

SmartCalc v2.0 项目教学文档

从编译运行到表达式引擎与绘图流程

OpenCode powered by ChatGPT

2026 年 3 月 19 日

目录

1 学习路线与项目全景	1
1.1 这个项目到底解决了什么问题	1
1.2 本教程的学习路线	2
1.3 开始之前你需要具备什么	3
本章小结	3
2 从零编译并运行项目	4
2.1 先建立一个足够用的 Qt 认识	4
2.2 在这个项目里, Qt 具体负责什么	5
2.3 先判断项目的真实构建入口	5
2.4 跨平台构建的通用思路	6
2.5 怎样在自己的系统上找到 Qt	7
2.5.1 Linux	7
2.5.2 macOS	8
2.5.3 Windows	8
2.6 分别在三种系统上怎样构建与运行	8
2.6.1 Linux 示例	8
2.6.2 macOS 示例	9
2.6.3 Windows 示例	9
2.7 为什么不把 Makefile 作为跨平台主线	9
2.8 编译报错时应该怎样定位	11
本章小结	11
3 理解 MVC: 程序是怎样被接起来的	13
3.1 为什么要先学架构, 而不是直接扎进按钮逻辑	13
3.2 MVC 到底是什么	13

3.2.1	为什么说 MVC 解决的是“变化分离”问题	14
3.3	MVC 在 SmartCalc 里是怎样落地的	15
3.4	从主程序入口观察 MVC 装配	15
3.5	View 层到底包含什么	16
3.5.1	主窗口 View	16
3.5.2	绘图窗口 Graph	17
3.6	Controller 为什么“薄”反而是优点	17
3.7	Model 层在这个项目里并不只有一个类	19
3.8	用一张图看清数据流	19
3.9	把两条典型调用链真正走一遍	20
3.9.1	调用链一：点击等号	20
3.9.2	调用链二：打开图窗	20
3.10	为什么这种 MVC 组织方式对学习者的尤其有价值	21
	本章小结	21
	本章练习	21
4	表达式引擎：从字符串到数值结果	22
4.1	为什么不能直接拿原始字符串硬算	22
4.2	第一步： FormatString 为什么存在	23
4.3	第二步：调度场算法如何把中缀表达式转成 RPN	24
4.3.1	优先级是怎么定义的	28
4.3.2	一元正负号是如何处理的	29
4.4	第三步：栈求值怎样得到最终结果	29
4.4.1	从测试中理解实现细节	31
	本章小结	33
	本章练习	33
5	函数绘图：同一套计算内核如何复用到图像显示	34
5.1	图窗打开时到底发生了什么	34
5.2	绘图本质上是“批量代值”	35
5.3	Graph 类怎样把点真正画出来	37
5.4	为什么有时候“图画不出来”其实不是程序坏了	39
	本章小结	39
6	测试、排错与继续扩展	40

6.1	测试覆盖了哪些核心能力	40
6.2	常见坑：不仅要知道是什么，还要知道怎么定位	41
6.2.1	构建层的坑	41
6.2.2	表达式层的坑	41
6.2.3	绘图层的坑	42
6.3	如果要继续扩展，这个项目适合怎么改	42
6.4	全文复习与综合练习	43
	本章小结	43
A	附录：项目中定义的类总览	44
A.1	类之间的整体关系	44
A.2	界面层类	47
A.2.1	View	47
A.2.2	Graph	48
A.3	控制层类	48
A.3.1	Controller	48
A.4	模型层类	49
A.4.1	Calculation	49
A.4.2	PlotGraph	49
A.4.3	ReversePolishNotation	50
A.4.4	Lexeme	50
A.5	辅助类	51
A.5.1	FormatString	51
A.6	一个很重要但不是项目自定义的类：QCustomPlot	51
	附录小结	52

Chapter 1

学习路线与项目全景

本教程面向想把一个 Qt 桌面项目真正“看懂、跑通、改动起来”的读者。你不需要先精通 Qt，也不需要先完整学过编译原理；但如果你会基本的 C++ 语法、知道函数和类是什么、并且愿意亲手运行命令和改几行代码，那么这份材料就能带你把 SmartCalc 项目从“能运行”推进到“知道为什么这样设计”。

1.1 这个项目到底解决了什么问题

很多初学者写计算器项目时，只做到两件事：界面按钮能点、四则运算能算。而 SmartCalc 多走了几步：

- 它不仅做普通算术，还支持科学计算函数，例如正弦、对数和平方根。
- 它支持变量 x ，因此不只是“输入一个数得一个结果”，而是能处理函数表达式。
- 它把同一套计算内核复用到绘图功能里，于是项目自然具备了“表达式求值 + 函数采样 + 图像展示”的完整闭环。
- 它使用模型-视图-控制器 (**Model-View-Controller, MVC**) 结构，把界面逻辑和数学逻辑分开，因而比“全部写在一个窗口类里”更适合学习工程化组织方式。

换句话说，SmartCalc 很适合作为一个教学项目，因为它同时覆盖了这几类知识：

1. Qt 桌面界面程序如何启动与组织；
2. 表达式如何从中缀写法转换成可计算形式；
3. 计算核心如何与图形界面解耦；
4. 一个功能不大的项目也能体现架构设计、测试和排错思维。

先建立整体印象

学一个项目时，最容易踩的坑不是“某一行代码没看懂”，而是“根本不知道自己现在看的这一段在整个系统里扮演什么角色”。所以我们第一章先建立地图，再进入细节。后面你看到某个类、某个命令、某条绘图链路时，就知道它位于整张地图的哪里。

节末自测 1

请先不用看源码，尝试回答三个问题：

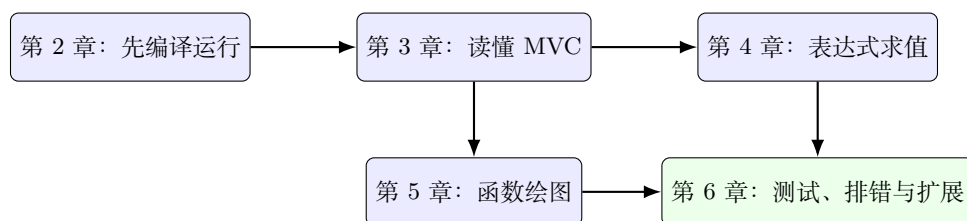
1. 如果一个项目支持表达式绘图，那么它为什么一定要支持变量 x ？
2. 为什么“图形界面 + 计算内核”这样的组合比单纯做一个命令行计算器更能练到工程能力？
3. 如果你以后想把这个项目改成命令行版，最希望哪些逻辑不要写死在界面层？

参考思路

关键在于：绘图需要对多个自变量取值重复计算；工程能力来自“同一能力被多个模块复用”；而命令行版与图形版都应该共用表达式解析与计算模块，而不是复制一份逻辑。

1.2 本教程的学习路线

本教程按“先跑通、再拆开、再举一反三”的顺序组织：



每一章各自解决的问题如下：

- 第 2 章解决“怎样在 Linux + 本地 Qt 环境中把项目正确编出来，并知道为什么推荐直接用 CMake”。
- 第 3 章解决“程序从 `main.cc` 启动后，View、Controller、Model 分别做什么”。
- 第 4 章解决“表达式为何要先格式转换，再做逆波兰表示法 (**Reverse Polish Notation, RPN**) 转换，最后栈求值”。
- 第 5 章解决“为什么绘图不是魔法，而是对表达式做一系列离散采样”。
- 第 6 章解决“测试覆盖了什么、常见坑在哪里、如果你继续扩展功能应先改哪里”。

阅读建议

如果你的目标是“先把项目用起来”，请按顺序读；如果你已经成功编译，只想重点理解算法，那么第 2 章可以略读，但建议至少记住 CMake 与 Makefile 的差异以及 Linux 下的构建注意事项。

节末自测 2

用自己的话复述本教程的主线：从用户点下一个按钮开始，到最终显示数值或绘制曲线，中间至少会经过哪三层角色？它们各自负责哪一类问题？

参考思路

可以概括为：界面层负责收集输入并约束格式，控制器负责转发请求，模型层负责把表达式转成可计算结构并给出数值结果；绘图时则在模型层重复计算多个 x 点，再由图窗显示。

1.3 开始之前你需要具备什么

本教程默认你具备如下前置知识：

- **必需前置**：会使用终端、知道如何进入目录、运行命令；能读基本的 C++ 类与成员函数；知道栈 (Stack) 是后进先出结构。
- **有帮助但不是必须**：用过 Qt Designer、知道信号与槽 (Signal / Slot) 的概念、接触过一点表达式求值算法。

如果你对“逆波兰表达式”完全陌生也没关系。本项目正好能让你理解一个非常常见的工程做法：用户写的是中缀表达式，程序内部计算的往往不是原样字符串。

开始动手前请记住一件事

不要把“项目能跑起来”误以为“项目已经看懂”。真正的理解，至少要包含三层：我会运行；我知道模块边界；我能指出如果要加新功能，应该优先修改哪一层。

本章小结

这一章你应该建立起了两个最重要的认识：

1. SmartCalc 不是单纯的按钮演示程序，而是一个把科学计算、表达式解析、绘图与 MVC 架构结合起来的教學型项目。
2. 学习路线应当从“先跑通工程”开始，再逐步过渡到“拆解架构”和“理解算法”。

本章练习

1. 试着用一句话描述这个项目最核心的卖点，不要超过 25 个字。
2. 假设你要向同学解释 MVC 在这个项目里的意义，请分别用“界面问题”“业务问题”“中介问题”三个短语去概括 View、Model、Controller。
3. 开放题：如果未来要增加“历史记录”功能，你觉得它更接近 View 层职责、Model 层职责，还是需要两边配合？请给出你的理由。

本章练习参考答案

第 1 题示例：支持科学计算和函数绘图的 Qt 计算器。第 2 题可概括为：View 负责界面交互，Model 负责表达式与数值逻辑，Controller 负责桥接调用。第 3 题没有唯一答案，但通常至少需要两边配合：界面负责展示与触发，模型或独立数据对象负责存储历史条目。

Chapter 2

从零编译并运行项目

这一章的目标非常明确：**让你在自己的机器上稳定地把 SmartCalc 编译出来，并且知道每一步为什么要这样做。**这里特别要避免一个常见误区：教程里出现过某台机器上的 Qt 路径，不代表所有电脑都必须照抄那个路径。真正应该学会的是**构建方法**，而不是死记某个具体目录。

在开始之前，先说一句会让初学者安心很多的话：**你不需要是 Qt 专家，依然可以读懂并构建这个项目。**如果你已经会基本的 C++，并且对 Qt 只有“看过一点、知道它能做 GUI”的程度，这其实已经足够。对本项目来说，你当前真正需要掌握的是：Qt 在这里扮演什么角色、构建时为什么必须找到 Qt、以及运行时哪些类属于 Qt 提供的界面基础设施。

2.1 先建立一个足够用的 Qt 认识

如果把 SmartCalc 暂时看成一个普通 C++ 项目，那么 Qt 可以理解成两部分能力的组合：

1. **界面框架能力**：帮你创建窗口、按钮、输入框、对话框、事件循环等桌面应用基础设施；
2. **工程工具链能力**：通过 moc、uic、资源系统和 CMake 集成，帮你把 .ui 界面文件、信号与槽机制以及资源文件编译进最终程序。

对只懂 C++、稍微了解过 Qt 的读者来说，本项目里最值得先认识的几个 Qt 概念是：

- **QApplication**：整个 GUI 程序的应用对象，负责启动事件循环。没有它，窗口程序就无法正常运转。
- **QMainWindow**：主窗口基类。SmartCalc 的 View 继承自它，所以它是整个计算器主界面的载体。
- **QDialog**：对话框基类。项目里的 Graph 继承自它，所以绘图窗口本质上是一个独立对话框。
- **QString**：Qt 自己的字符串类型。界面层大量用它，是因为 Qt 控件 API 天然围绕 QString 设计。
- **信号与槽 (Signal / Slot)**：Qt 的事件响应机制。用户点击按钮后，最终就是某个槽函数被调用。
- **.ui 文件**：由 Qt Designer 或类似工具描述界面的 XML 文件，编译时会被 Qt 的工具转换成可用的界面代码。

你现在不需要一下子学完 Qt

对 SmartCalc 这种项目，最合理的学习策略不是“先把整本 Qt 教程啃完”，而是先掌握和当前项目直接有关的那一小部分：程序如何启动、窗口类如何继承、按钮点击如何落到槽函数、QString 为什么频繁出现、.ui 文件为什么能参与构建。只要这几个点通了，后续读代码就不会慌。

2.2 在这个项目里，Qt 具体负责什么

从代码结构上看，Qt 在 SmartCalc 中主要承担四件事：

1. 提供主窗口和绘图窗口这两类 GUI 容器；
2. 提供按钮、输入框、显示框、自旋框等现成控件；
3. 提供 QString、QVector 等常用类型，方便界面和绘图库协作；
4. 通过 CMake 的自动处理能力，把 Q_OBJECT、.ui 和资源文件接入编译流程。

如果你回头看 src/CMakeLists.txt，会发现这几行尤其关键：

- `set(CMAKE_AUTOUICON)`：自动处理 .ui 文件；
- `set(CMAKE_AUTOMOC)`：自动处理带有 Q_OBJECT 的类；
- `set(CMAKE_AUTORCC)`：自动处理 Qt 资源文件；
- `find_package(...Widgets...PrintSupport)`：告诉 CMake 这个项目依赖哪些 Qt 组件。

对于初学者来说，这里最值得形成的理解是：**Qt 不只是一个头文件库，它还有一套自己的代码生成和资源处理流程**。这也是为什么“能不能正确找到 Qt”会直接影响构建是否成功。

Qt 快速自测

1. 为什么一个普通 C++ 项目只靠编译器就能编，而 Qt 项目往往还需要额外的自动处理步骤？
2. 在本项目里，View 和 Graph 分别更接近 QMainWindow 还是 QDialog 这样的哪类角色？

参考思路

第 1 题的关键在于：Qt 除了普通 C++ 源码外，还涉及 Q_OBJECT、.ui、资源文件等需要工具链额外处理的内容。第 2 题里，View 更接近主窗口角色，而 Graph 更接近对话框角色。

2.3 先判断项目的真实构建入口

判断一个 C++ 项目如何构建，最直接的方法是看仓库里的构建描述文件。这个项目的核心构建脚本在 src/CMakeLists.txt，而不是仓库根目录。

```
1 cmake_minimum_required(VERSION 3.5)
```

```
2
```

```

3  project(view VERSION 0.1 LANGUAGES CXX)
4
5  set(CMAKE_INCLUDE_CURRENT_DIR ON)
6
7  set(CMAKE_AUTOUIC ON)
8  set(CMAKE_AUTOMOC ON)
9  set(CMAKE_AUTORCC ON)
10
11 set(CMAKE_CXX_STANDARD 17)
12 set(CMAKE_CXX_STANDARD_REQUIRED ON)
13
14 find_package(QT NAMES Qt6 Qt5 REQUIRED COMPONENTS Widgets)
15 find_package(Qt${QT_VERSION_MAJOR} REQUIRED COMPONENTS Widgets)
16 find_package(Qt${QT_VERSION_MAJOR} COMPONENTS PrintSupport REQUIRED)
17
18 include_directories(${CMAKE_SOURCE_DIR}/view)
19 include_directories(${CMAKE_SOURCE_DIR}/model)
20

```

从这段配置里你至少要读出三个关键信息：

1. 这是一个 CMake 项目，而不是 qmake 项目，因为核心构建入口是 `CMakeLists.txt`。
2. 项目通过 `find_package` 查找 Qt 的 `Widgets` 和 `PrintSupport` 组件，所以编译时必须让 CMake 找到本机 Qt 安装路径。
3. 可执行目标的逻辑名叫 `view`，但最终输出名被设成了 `SmartCalc`。

这就决定了最稳妥的主线构建方式：直接调用 CMake，并在需要时显式告诉它 Qt 装在哪里。

对 Qt 初学者来说，这里还有一个很重要的认识：`find_package(Qt...)` 不是“某种可选优化”，而是项目构建的一部分核心逻辑。因为只要这些 Qt 组件没被正确发现，`QMainWindow`、`QDialog`、`QString`、`.ui` 文件对应的构建过程就都无法顺利接起来。

节末自测 1

1. 如果本机安装了多个 Qt 版本，CMake 是通过什么机制定位到正确版本的？
2. 为什么输出目标名可能与 CMake 中的逻辑 `target` 名不同？

参考思路

第 1 问的关键是 `find_package` 与 CMake 的查找路径变量，例如 `CMAKE_PREFIX_PATH`。第 2 问的关键是 `set_target_properties(...OUTPUT_NAME...)` 可以把生成物名称和内部 `target` 名分开。

2.4 跨平台构建的通用思路

先不要急着记某个路径。无论你使用的是 Windows、macOS 还是 Linux，核心步骤其实都是同一套：

1. 确认 CMake、C++ 编译器和 Qt 都已安装；
2. 找到当前机器上的 Qt 安装前缀；
3. 用 `cmake -S` 指向源码目录，用 `-B` 指向构建目录；
4. 必要时通过 `CMAKE_PREFIX_PATH` 告诉 CMake 去哪里找 Qt；
5. 调用 `cmake --build` 完成编译；
6. 根据当前平台的产物形式运行程序。

一个通用模板如下：

```
1 cmake -S src -B src/build -DCMAKE_PREFIX_PATH="< 你的 Qt 安装前缀 >"
2 cmake --build src/build
```

这里最重要的不是尖括号里的具体字符串，而是你要知道：那一项必须换成你自己电脑上的 Qt 路径。

如果你已经用过一点 Qt Creator，也可以把这里的过程理解成“手工复现 Kit 的核心信息”：

- 编译器是谁；
- Qt 安装前缀在哪里；
- 生成器和构建目录是什么；
- 最终要链接哪些 Qt 模块。

也就是说，即便你不是 Qt 专家，只要你能把“Kit 里包含的关键信息”映射回 CMake 命令，就已经足够自己完成大多数桌面 Qt 项目的基础构建。

为什么推荐这样写

`-S` 和 `-B` 明确区分“源码目录”和“构建目录”，不容易把中间产物散落到源代码里。而且当你以后写脚本、写 CI 或换电脑复现时，这种写法更容易复制、阅读和排错。

2.5 怎样在自己的系统上找到 Qt

Qt 的安装位置和编译套件 (Kit) 会因平台与安装方式不同而变化，所以这里给你的应该是识别方法和常见示例，而不是唯一答案。

2.5.1 Linux

Linux 上常见的几种情况包括：

- 使用官方安装器时，常见路径类似 `$HOME/Qt/6.7.0/gcc_64`；
- 也可能安装在别的用户目录、自定义目录或系统目录下；
- 如果不是官方安装器，而是系统包管理器安装，布局可能又会不同。

对 Linux 来说，一个很实用的判断办法是：检查你的 Qt 前缀目录下是否存在 `lib/cmake/Qt6/Qt6Config.cmake`。

2.5.2 macOS

macOS 上常见情况通常有两类：

- 使用 Qt 官方安装器，路径常类似 `/Users/< 用户名 >/Qt/6.7.0/macos`；
- 使用 Homebrew 或其他方式安装时，路径可能位于包管理器目录中。

和 Linux 一样，关键仍然是找到能让 CMake 发现 `Qt6Config.cmake` 的前缀路径，而不是机械照抄某个目录名。

2.5.3 Windows

Windows 上最常见的情况是 Qt 官方安装器。常见目录类似：

- `C:/Qt/6.7.0/msvc2022_64`
- `C:/Qt/6.7.0/mingw_64`

这里要特别注意：**路径不仅和 Qt 版本有关，还和你选择的编译器套件有关**。如果你使用 MSVC，就不要把 MinGW 的套件路径传给 CMake；反过来也一样。

节末自测 2

1. 为什么不能把别人电脑上的 `$HOME/Qt/...` 或 `C:/Qt/...` 直接当成自己机器的固定答案？
2. 如果你已经找到了某个疑似 Qt 目录，最值得优先检查哪个文件，来确认它是不是正确前缀？

参考思路

第 1 题因为安装器、操作系统、用户名、自定义目录和编译器套件都可能不同。第 2 题的关键是去找 `Qt6Config.cmake`，它通常能帮助你确认当前目录是否适合作为 CMake 的查找前缀。

2.6 分别在三种系统上怎样构建与运行

下面给出三种平台下的教学型示例。请注意，这些命令中的 Qt 路径都只是**示例形式**，真正执行时要换成你自己电脑上的路径。

2.6.1 Linux 示例

```
1 cmake -S src -B src/build -DCMAKE_PREFIX_PATH="< 你的 Linux Qt 前缀 >"
2 cmake --build src/build -j
3 ./src/build/SmartCalc
```

如果你的 Qt 恰好装在用户目录下，那么这个前缀可能长得像 `$HOME/Qt/6.7.0/gcc_64`；但这只是某一种常见情况。

2.6.2 macOS 示例

```
1 cmake -S src -B src/build -DCMAKE_PREFIX_PATH="< 你的 macOS Qt 前缀 >"
2 cmake --build src/build -j
3 open src/build/SmartCalc.app
```

如果生成的是应用包 `.app`，那么用 `open` 打开通常比较方便；如果你需要直接运行内部二进制，也可以进入应用包内部的可执行文件路径。

2.6.3 Windows 示例

```
1 cmake -S src -B src/build -DCMAKE_PREFIX_PATH="< 你的 Windows Qt 前缀 >"
2 cmake --build src/build --config Release
3 .
4 \src\build\Release\SmartCalc.exe
```

在 Windows 上，是否需要 `--config Release` 取决于你使用的生成器和工具链；如果是多配置生成器，这个参数通常更常见。

一个跨平台都成立的判断原则

当你不确定运行产物在哪时，先不要凭印象猜。更稳妥的办法是观察构建目录里实际生成了什么：Linux 常见的是可执行文件，macOS 常见的是 `.app`，Windows 常见的是 `.exe` 或带配置子目录的产物。

2.7 为什么不把 Makefile 作为跨平台主线

项目里确实还有一个 `src/Makefile`，但它更适合当作“附加脚本集合”，而不是跨平台主构建入口。请看其中一部分内容：

```
1 CC=g++
2 CFLAGS=-Wall -Werror -Wextra -std=c++17
3
4 all: clean check_style test gcov_report install uninstall dvi dist
5
6 cmake:
7     @mkdir -p build && cd build && cmake ..
8
9 install: cmake
10    @echo "\033[46mInstallation\033[0m"
11    @cd build && make
12    @echo "\033[46mApplication installed\033[0m"
13    @make dvi
14    @cd build && open SmartCalc.app
15
16 uninstall: clean
17    @rm -rf build
18    @rm -rf ../dist
```

```

19     @echo "\033[46mApplication removed\033[0m"
20
21 test: clean
22     $(CC) $(CFLAGS) --coverage model/tests/*.cc model/*.cc -lgtest -lgtest_main -pthread -o test
23     ./test
24
25 gcov_report: test
26     @lcov -t "test_calculation" -o calculation.info -c -d . --ignore-errors mismatch,inconsistent
27     @lcov -e calculation.info '*/src/model/*.cc' -o calculation_filtered.info --ignore-errors
28     ↪ mismatch,inconsistent
29     @genhtml -o ../gcov_report calculation_filtered.info --ignore-errors mismatch,inconsistent
30     @open ../../gcov_report/index.html
31
32 dvi:
33     @open ../../dvi_report/html/index.html
34
35 dist:
36     @mkdir -p ../dist
37     @cp -R build/SmartCalc.app ../dist
38     @cd ../dist && tar cvzf SmartCalc.tar *
39     @cd ../dist && rm -rf SmartCalc.app README.md

```

这里至少有三处值得注意：

1. `install` 目标里用了 `open SmartCalc.app`，明显偏向 macOS；
2. `dist` 目标把产物当成 `SmartCalc.app` 处理，也更像 macOS 应用包；
3. 这份 Makefile 没有把 Windows 的典型 workflow 纳入主线，因此不适合作为一份真正的跨平台通用教程入口。

所以更准确的说法应该是：Makefile 不是完全不能用，而是不适合作为跨平台新手教程的第一条路径。对于编译和运行 Qt 程序，直接讲 CMake 更统一、更稳定。

节末自测 3

如果你执行 `make install` 后遇到和 `open`、`.app` 或平台命令有关的报错，这类错误更可能说明什么？

- A. Qt 没装好。
- B. C++ 编译器没装好。
- C. Makefile 内部带有平台偏向。
- D. 代码一定有语法错误。

请说明你的判断和理由。

参考思路

更可能是 C。因为 `open` 和 `.app` 本身就带有明显的平台语义，它们和 Qt 是否安装、源码是否语法正确，不属于同一层问题。

2.8 编译报错时应该怎样定位

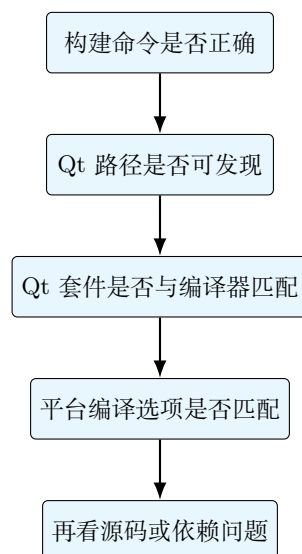
编译型项目的排错，最忌讳一上来就“全部重装”。更好的习惯是按层排查：

1. 先确认命令是否指向对的源码目录和构建目录；
2. 再确认 CMake 是否能找到 Qt；
3. 再确认当前 Qt 套件是否和编译器匹配；
4. 最后再看是否是平台相关编译选项或源码问题。

这个项目曾经出现过一个非常典型的跨平台编译坑：

- 源文件中无条件加入了 `-Wa,-mbig-obj` 选项；
- 这个选项更适合 MinGW / Windows 场景；
- Linux 下的汇编器不认识它，于是会报类似“`as: unrecognized option '-mbig-obj'`”的错误。

正确思路不是怀疑 Qt，也不是怀疑外层脚本，而是回到 `src/CMakeLists.txt` 看这个选项是否应该受平台条件保护。现在项目已经改成只有在 MINGW 条件下才添加这个选项，因此 Linux 构建能够通过；而这个案例本身也很适合提醒你：跨平台项目经常不是“代码不会编”，而是“某个选项只适用于特定平台”。



新手非常常见的误区

看到外层脚本、批处理文件或 Python 脚本报错，不代表这些脚本本身就是问题根源。很多时候，它们只是“调用了外部构建命令的壳”，真正失败的仍然是 CMake、编译器、Qt 查找路径或平台选项。排错时要先区分“谁在执行命令”和“谁真正失败了”。

本章小结

这一章的核心结论有三条：

1. 主构建入口应优先选择 CMake，因为它比仓库中的 Makefile 更适合写成跨平台教程。
2. Qt 的关键不在于记住某个固定路径，而在于学会找到自己机器上的 Qt 前缀，并让 `find_package` 能发现它。
3. 构建报错时要按“命令 -> Qt 路径 -> 套件匹配 -> 平台选项 -> 源码”这条路径逐层排查。

本章练习

1. 请自己在终端中解释这条命令每个参数的作用：`cmake-Ssrc-Bsrc/build-DCMAKE_PREFIX_PATH=...`。
2. 动手题：把构建目录换成 `src/build-debug`，重新执行构建命令，确认你已经理解 `-B` 的作用。
3. 分析题：如果一位同学使用 Windows + MSVC，另一位同学使用 Linux + GCC，那么他们真正需要变化的是哪一部分信息，哪些构建思路完全相同？

本章练习参考答案

第 1 题应能解释源码目录、构建目录和 CMake 变量注入。第 2 题的关键结果是：源码目录不变，构建产物切换到新目录。第 3 题真正变化的是 Qt 前缀路径、编译器套件和最终产物形式；而相同之处是：都通过 CMake 指向源码与构建目录，并让 CMake 正确找到 Qt。

Chapter 3

理解 MVC：程序是怎样被接起来的

成功编译只是第一步。真正重要的是：**程序启动后，表达式输入、计算和绘图请求分别流向哪里？**如果这个问题答不出来，那么以后你想加功能时就会非常痛苦，因为你根本不知道要改哪层。

这一章会把 MVC (**Model-View-Controller**) 讲得更完整一些。因为对很多只懂 C++、只稍微接触过 Qt 的读者来说，最大的困难往往不是“某个函数没看懂”，而是“我虽然听过 MVC 这个词，但它到底为什么要存在，它在这个项目里又具体落在哪些类上”。

3.1 为什么要先学架构，而不是直接扎进按钮逻辑

初学者读 GUI 项目时，特别容易一上来就去看“按钮点击后发生了什么”。这种做法短期看起来很直接，但有一个明显问题：你会很快被局部细节淹没，最后只记住了一堆槽函数名字，却还是说不清整个程序为什么这样组织。

真正更省力的顺序通常是：

1. 先知道项目被分成了哪几层；
2. 再知道每一层各自负责哪类问题；
3. 最后才进入具体点击流程和类实现。

对于 SmartCalc 来说，这种分层思路并不是“锦上添花”，而是整个项目最值得学习的工程部分之一。因为它不是把所有逻辑都塞进一个主窗口类里，而是明确使用了 MVC 结构。

先有地图，再看街道

把源码阅读想成看一座城市。如果你还没看地图，就直接随机走进几条街，很容易迷路；但如果你先知道商业区、居住区、交通枢纽分别在哪里，再去看某一条街，就会轻松得多。MVC 对读代码的帮助，和地图对城市导航的帮助很像。

3.2 MVC 到底是什么

MVC 的全称是 **Model-View-Controller**，中文通常翻译为“模型-视图-控制器”。

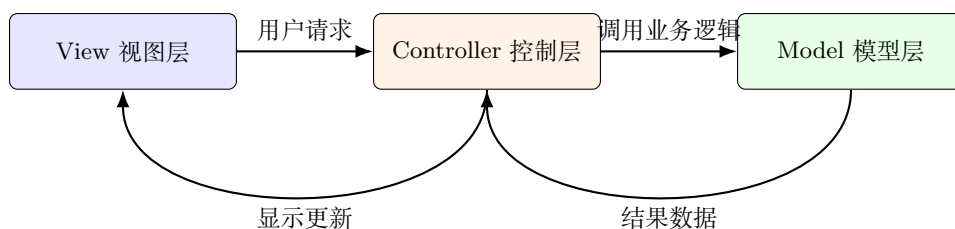
它不是某个只属于 Qt 的神秘术语，而是一种非常经典的软件设计思想。它想解决的核心问题其实很朴素：**不要让界面显示、业务逻辑和交互调度搅在一起。**

如果把一个程序完全不分层地堆在一起，通常会出现这些问题：

- 窗口类越来越大，既负责显示按钮，又负责判断输入是否合法，还负责真正计算结果；
- 一旦你想复用业务逻辑做测试、命令行版或新界面，就会发现大量代码被绑死在 GUI 上；
- 改一个功能时很容易波及不该受影响的区域；
- 新读者很难快速分辨“哪部分是业务规则，哪部分只是界面细节”。

MVC 的基本思路就是把这些责任拆开：

1. **Model，模型层：**负责真正的业务逻辑和数据处理。对 SmartCalc 来说，就是表达式解析、数值计算、绘图坐标生成这一类工作。
2. **View，视图层：**负责界面显示和用户交互。对 SmartCalc 来说，就是主窗口、绘图窗口、按钮、显示框以及用户点击后的界面响应。
3. **Controller，控制层：**负责在 View 和 Model 之间做连接与转发。它把界面层提出的请求送到模型层，再把结果回传给界面层。



3.2.1 为什么说 MVC 解决的是“变化分离”问题

理解 MVC 最好的方式之一，是去想“哪些东西以后最容易变化”。

在这个项目里，至少有三类变化来源：

- 界面可能变化：按钮样式、布局、窗口、图窗交互方式都可能改；
- 业务逻辑可能变化：新增函数、修改运算优先级、改进绘图采样方式；
- 交互流程可能变化：某些操作是否要多一步检查、是否加入日志、是否加入中间状态控制。

如果这些变化全都混在同一个类里，后续维护代价会越来越高；而 MVC 的目标就是让不同类型的变化尽量落在不同层里。

一个很常见的误解

MVC 不是说“三层之间完全没有关系”，也不是说“只要把文件夹叫成 model、view、controller 就算架构好”。真正重要的是：每一层承担的责任是否清楚，边界是否稳定，修改某一类问题时是否能优先在某一层内完成。

节末自测 1

1. MVC 想解决的核心问题是什么？
2. 为什么“把所有代码都塞进主窗口类里”在小项目里也仍然可能是个坏主意？

参考思路

第 1 题的关键词是责任分离、变化分离和降低耦合。第 2 题的关键在于：哪怕项目不大，只要界面逻辑和业务逻辑混在一起，测试、复用、扩展和排错都会明显变难。

3.3 MVC 在 SmartCalc 里是怎样落地的

理解一个架构最好的办法，不是背定义，而是把它落实到当前项目的具体类上。对 SmartCalc 来说，可以先建立下面这个映射关系：

- **View:** View 主窗口类，以及负责图像显示的 Graph 对话框；
- **Controller:** Controller；
- **Model:** Calculation、PlotGraph、ReversePolishNotation、Lexeme 等围绕表达式和绘图数据工作的类；
- **边界辅助:** FormatString 负责把界面层的字符串表示整理成模型更容易处理的内部形式。

需要注意的是，真实工程中的分层从来不会像教科书图示那样绝对“纯净”。例如 FormatString 放在 view 目录下，但它又明显服务于模型前的预处理。对学习者来说，最重要的不是纠结它“应该百分之百属于哪一层”，而是看清它在调用链里的作用：它站在界面输入和模型解析之间，负责做表示转换。

3.4 从主程序入口观察 MVC 装配

先看项目入口文件 src/main.cc：

```
1  #include <QApplication>
2
3  #include "controller/controller.h"
4  #include "view/view.h"
5
6  int main(int argc, char *argv[]) {
7      s21::Calculation calculation;
8      s21::Controller controller(&calculation);
9
10     QApplication a(argc, argv);
11     a.setWindowIcon(QIcon(":/pics/calculator_icon.png"));
12     View view(nullptr, &controller);
13     view.show();
14     view.setWindowTitle("SmartCalc");
15
16     return a.exec();
17 }
```

这段代码虽然很短，但它已经把整个项目的骨架说清楚了：

1. 先创建 `s21::Calculation`，也就是核心计算模型；
2. 再用这个模型构造 `s21::Controller`，也就是界面和模型之间的中介；
3. 然后创建 `View`，把控制器指针传进去；
4. 最后进入 Qt 事件循环 `a.exec()`，此后程序主要靠用户操作驱动。

这就是一个非常典型的 MVC 起手式：先有 Model，再有 Controller，最后把 Controller 交给 View。

对 Qt 初学者来说，这里还有两个额外值得注意的点：

- `QApplication` 是 GUI 程序的“总入口对象”，它负责事件循环；
- `View` 并不是一个普通 C++ 类实例，而是一个真正会显示在屏幕上的 Qt 主窗口。

为什么这种连接方式适合教学

因为它特别直观。你只看入口就能知道：`View` 不是凭空自己算结果，而是通过 `Controller` 间接使用 `Model`。这个边界在项目早期就画清楚了，后面读代码会轻松很多。

节末自测 2

如果把 `Calculation` 直接写到 `View` 内部，让窗口类自己既处理按钮、又解析表达式、又计算结果，会产生什么问题？请至少说出两点。

参考思路

典型问题包括：界面层与业务层耦合过紧；以后想复用计算逻辑做命令行版或测试时不方便；窗口类会越来越臃肿，难以维护和排错。

3.5 View 层到底包含什么

很多人在第一次接触 MVC 时，会把 View 理解得过于狭窄，以为它只是“把东西画出来”。在 `SmartCalc` 里，View 层当然承担显示职责，但它还承担了相当多的交互约束职责。

3.5.1 主窗口 View

阅读 `src/view/view.h`，你会发现这个类里有大量槽函数：数字按钮、运算符按钮、函数按钮、括号、回退、等号、图窗打开等等。

它还维护了一组状态标志，例如：

- `num_clicked_`
- `point_clicked_`

- operator_clicked_
- x_clicked_
- e_clicked_
- open_parenthesis_clicked_

这些标志说明了一件很重要的事：**View** 不只是“把按钮画出来”，它还承担了输入合法性约束。

例如：

1. 不允许一个数字词元里无限乱点小数点；
2. 某些位置不能直接接乘除号；
3. 平方根、函数名、幂运算后面会自动补上左括号；
4. 图窗打开前会先检查当前表达式是否满足基本合法性。

这和一些“强健解析器”项目不同。SmartCalc 的一个鲜明特点是：**很多非法输入是在界面层就被尽量拦住的，而不是完全留给模型层兜底。**

3.5.2 绘图窗口 Graph

Graph 也是 View 层的一部分。它虽然不是主窗口，但它负责图像显示、坐标范围控制、曲线刷新和图窗清理。换句话说，它是专门负责“图像视图”的界面类。

因此，在这个项目里，View 层并不只对应一个类，而是对应一组围绕“显示和交互”展开的类。

一个很容易混淆的点

View 层可以做输入约束，但这不等于说业务逻辑全在 View 里。更准确的说法是：View 负责让用户更难输入明显无效的内容，而真正的数学求值和绘图坐标生成仍然属于模型层。

3.6 Controller 为什么“薄”反而是优点

来看控制器 src/controller/controller.h 中最核心的两个接口：

```

15 class Controller {
16 public:
17     /**
18      * @brief Constructs a Controller with a given Calculation model.
19      * @param calculation pointer to a Calculation object used for parsing
20      * expressions and computing results.
21      */
22     Controller(Calculation *calculation) : model_(calculation){};
23
24     /**
25      * @brief Default destructor.
26      */

```

```

27 ~Controller() = default;
28
29 /**
30  * @brief Calculates the result of a mathematical expression.
31  * @param expression string representing the mathematical expression to be
32  * evaluated.
33  * @param x_value x-value to substitute into the expression during evaluation.
34  * @return computed result as a long double.
35  */
36 long double Calculate(std::string expression, double x_value) {
37     model_->Parse(expression, x_value);
38     return model_->GetResult();
39 }
40
41 /**
42  * @brief Calculates the x and y coordinates for graphing based on the given
43  * expression.
44  * @param expression string representing the mathematical expression to be
45  * evaluated.
46  * @param x_range pair of long doubles representing the range of x-values.
47  * @return pair of vectors containing the x-coordinates and corresponding
48  * y-coordinates.
49  */
50 std::pair<std::vector<double>, std::vector<double>> CalculateGraphCoordinates(
51     std::string &expression, std::pair<long double, long double> x_range) {
52     return plot_.Calculate(expression, x_range);
53 }
54
55 private:
56     Calculation
57         *model_; ///< pointer to the Calculation model for expression evaluation
58     PlotGraph plot_; ///< instance of PlotGraph for calculating graph coordinates
59

```

很多初学者第一次看到这种控制器，会觉得它“好像没做什么”。其实这正是它的价值。

它的工作可以概括成两件事：

- Calculate(...) 把表达式和 x 值交给模型，再取回结果；
- CalculateGraphCoordinates(...) 把表达式和坐标范围交给绘图计算对象，再返回坐标对。

这类控制器常被称为“薄控制器”。这里的“薄”不是贬义，而是说它没有把本不属于自己的业务硬塞进来。

它的优点在于：

1. 界面层不用知道模型内部细节；
2. 模型层也不用感知 Qt 控件；
3. 如果以后控制器要补日志、参数检查、权限控制或异常包装，就有一个自然的位置去放，而不必污染 View 或 Model。

节末自测 3

为什么 Controller 同时持有 Calculation 和 PlotGraph 两类能力，而不是让 View 自己直接去创建绘图计算对象？

参考思路

因为绘图坐标计算仍然属于业务逻辑，不属于界面逻辑。只要某个功能本质上是在“根据表达式算数据”，它就更接近模型或控制器这边，而不是窗口控件本身。

3.7 Model 层在这个项目里并不只有一个类

很多教材用极简例子介绍 MVC 时，会把 Model 画成单一对象。但在真实项目里，模型层往往是一组彼此协作的业务类。SmartCalc 也是这样。

在这个项目里，模型层至少可以拆成几块：

- Calculation：负责表达式求值；
- ReversePolishNotation：负责把中缀表达式转换成后缀序列；
- Lexeme：负责表示表达式中的最小语义单元；
- PlotGraph：负责生成绘图用的坐标点；
- FormatString：负责把界面字符串预处理成模型更容易解析的形式。

这种拆分特别值得初学者学习，因为它体现了一个重要思维：复杂业务逻辑不一定塞进一个“超级类”，而是拆成多个更小、职责更清楚的类协作完成。

3.8 用一张图看清数据流

这一章最值得记住的，是下面这张结构图。它把项目的静态角色和主要数据流串在一起：

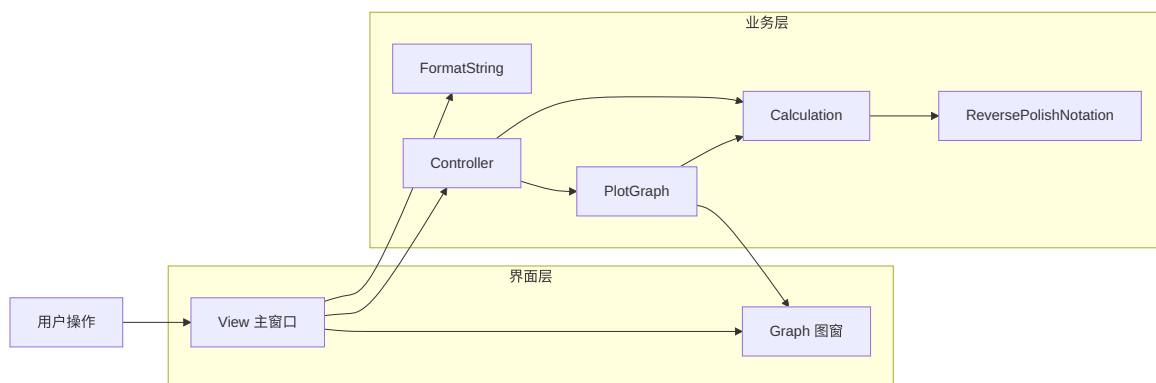


图 3.1: SmartCalc 模块总览

阅读这张图时，建议重点观察三件事：

1. `View` 和 `Graph` 都属于界面层，但一个负责主交互，一个负责图像展示；
2. `Controller` 位于中间，负责把“界面请求”变成“模型调用”；
3. `FormatString`、`ReversePolishNotation`、`Calculation`、`PlotGraph` 虽然功能不同，但都围绕“如何把表达式变成可计算数据”服务。

3.9 把两条典型调用链真正走一遍

理解架构最有效的方式之一，是把一条真实调用链完整讲清楚。这里选两条最典型的链路：点击等号，和打开图窗。

3.9.1 调用链一：点击等号

当用户点击 `=` 时，大体会发生这些事：

1. `View` 先检查当前表达式在界面层是否基本合法，例如括号是否闭合、末尾是否停在运算符上；
2. 如果基本合法，就把当前的 `QString` 交给 `FormatString` 做预处理；
3. 然后通过 `Controller` 请求计算；
4. `Calculation` 内部调用 `ReversePolishNotation` 做表达式结构转换；
5. 模型层完成数值求值后，把结果返回给控制器，再回到视图层；
6. `View` 再把结果整理成适合显示的字符串，并更新显示框。

这条链路最值得记住的一点是：**`View` 不直接做数学运算，`Model` 不直接操作界面显示。**

3.9.2 调用链二：打开图窗

当用户请求绘图时，流程与等号有相似之处，但会多出“批量采样”这一步：

1. `View` 打开或激活 `Graph` 图窗；
2. 它同样先检查表达式是否满足基本合法性；
3. 然后把表达式送入 `FormatString`；
4. 再通过 `Controller` 调用 `PlotGraph`；
5. `PlotGraph` 在很多个 `x` 点上复用 `Calculation` 计算对应的 `y`；
6. 最终生成的坐标向量被送回 `Graph`，由图窗显示出来。

这条链路说明了另一个架构优点：**同一个计算核心既能服务单次求值，也能服务绘图采样。**

阅读源码的顺序建议

如果你已经被 `view.cc` 的大量代码吓到了，可以先不要硬啃整文件。先按“入口 -> 控制器 -> 等号流程 -> 图窗流程 -> 其他按钮限制”的顺序看，会清楚很多。

3.10 为什么这种 MVC 组织方式对学习 者尤其有价值

对课程项目或练手项目来说,很多人会觉得“能跑就行,没必要讲架构”。但恰恰相反,像 SmartCalc 这种体量适中的项目,正是学习架构边界最好的材料。

因为它足够小,小到你能看完整条调用链;又足够完整,完整到能让你看到:

- 输入约束放在界面层是什么感觉;
- 业务逻辑独立出来后,测试为什么会更容易;
- 绘图功能如何复用已有计算核心;
- 控制器为什么即使不复杂,也依然有存在价值。

如果你以后要给这个项目加功能,例如历史记录、键盘输入、更多函数、自动缩放图像,那么 MVC 的好处会更明显:你至少知道应该先从哪一层开始想,而不是在整个仓库里盲目乱改。

本章小结

本章最核心的收获是:

1. MVC 的本质是责任分离和变化分离,不是一个只用于背定义的缩写;
2. 在 SmartCalc 中,View 负责显示与交互约束,Controller 负责中介转发,Model 负责表达式与绘图数据逻辑;
3. 这个项目的 Model 并不是一个单独类,而是一组围绕表达式求值和绘图坐标生成协作的类;
4. 真正理解架构的最好办法,是沿着“等号流程”和“绘图流程”把完整调用链走一遍。

本章练习

1. **复述题:** 请用自己的话解释 MVC 在这个项目里分别对应哪些类,以及它想解决什么问题。
2. **对比题:** 如果把 Calculation、按钮输入限制、图窗显示和绘图采样全部塞进 View 一个类里,短期和长期分别会有什么代价?
3. **调用链题:** 请分别口头复述“点击等号”和“打开图窗”这两条路径上,数据经过了哪些类。
4. **扩展题:** 如果你要给项目加一个“历史记录”功能,你觉得它首先会影响 View、Controller、Model 中的哪一层?是否需要多层配合?

本章练习参考答案

第 1 题的关键是把 MVC 落到具体类,而不是停留在抽象名词。第 2 题的关键是耦合、复用困难、测试困难和窗口类膨胀。第 3 题应能说出从 View 出发,经由 Controller 到模型层,再回到界面层的完整过程。第 4 题通常不是只影响单层:界面层负责入口和展示,控制层负责连接,模型层或独立数据结构负责记录与组织历史数据。

Chapter 4

表达式引擎：从字符串到数值结果

这一章是整份教程的核心。只要你真正理解了这一章，SmartCalc 最重要的业务逻辑就已经掌握了大半。

4.1 为什么不能直接拿原始字符串硬算

用户在界面上看到并输入的是人类友好的中缀表达式 (**Infix Expression**)，例如：

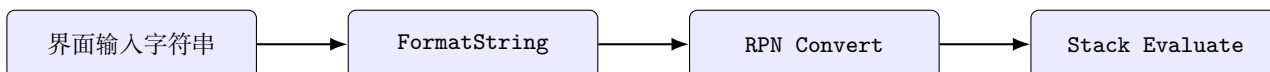
```
1 sin(x)+pow(2,3)-log(100)
```

但是对程序来说，这样的字符串有几个麻烦：

1. 函数名是多字符的，逐字符扫描时不方便统一处理；
2. 中缀表达式存在优先级和括号嵌套，直接边读边算会很乱；
3. 一元正负号、科学计数法中的 **e**、变量 **x** 等情况混在一起，如果没有中间表示，逻辑很容易缠成一团。

因此 SmartCalc 采用了三步走策略：

1. 先做 **格式转换**，把完整函数名压缩为内部更易处理的表示；
2. 再做 **逆波兰表示法 (Reverse Polish Notation, RPN)** 转换；
3. 最后用 **栈求值** 计算最终结果。



节末自测 1

为什么很多表达式求值程序都会先引入某种“中间表示”，而不是直接把用户输入的一整串文字硬塞进一个超级大的 if-else 里？

参考思路

因为中间表示能把“词法处理”“运算符优先级”“真正求值”这几个问题拆开。拆开后更容易测试、更容易维护，也更容易支持新操作符。

4.2 第一步：FormatString 为什么存在

请看 `src/view/format_string.cc`:

```
1  #include "format_string.h"
2
3  namespace s21 {
4
5  void FormatString::Convert() {
6      QString::iterator it = q_str_.begin();
7      size_t iter_move = 0;
8
9      while (it != q_str_.end()) {
10         iter_move = 1;
11
12         if (*it == 'l') {
13             it++;
14
15             if (*it == 'n') {
16                 basic_str_ += "l";
17
18             } else {
19                 basic_str_ += "L";
20                 iter_move = 2;
21             }
22
23         } else if (*it == 'a') {
24             it++;
25
26             basic_str_ += static_cast<char>(it->unicode() - 32);
27             iter_move = 3;
28
29         } else if (*it == 's' || *it == 'c' || *it == 't') {
30             basic_str_ += static_cast<char>(it->unicode());
31             iter_move = 3;
32
33         } else {
34             basic_str_ += static_cast<char>(it->unicode());
35         }
36
37         it += iter_move;
38     }
39 }
40
41 } // namespace s21
```

这段代码背后的思想非常值得初学者体会：界面给用户看的是“可读形式”，模型处理的是“方便机器解析的内部形式”。

它的映射关系大致如下：

界面字符串	内部表示	说明
ln(	l(	自然对数
log(	L(	常用对数
sin(	s(	正弦
cos(	c(	余弦
tan(	t(	正切
asin(	S(	反正弦
acos(	C(	反余弦
atan(	T(	反正切
sqrt(	r(	平方根

这种设计带来的好处是：

1. RPN 转换器只需要识别单字符运算符和函数码，逻辑更统一；
2. 模型层不必承担“完整英文函数名识别”这件事；
3. 界面显示与内部实现解耦，日后如果你想把按钮文字换成本地化语言，也不必重写整个解析器。

这里特别容易误解

模型层并不是直接解析 `sin(x)` 这样的完整英文串，而是更偏向解析 `s(x)` 这种压缩形式。也就是说，`FormatString` 不是可有可无的“美化步骤”，而是当前工程里实际的解析前置步骤。

节末自测 2

如果你直接跳过 `FormatString`，把 `"sin(1)"` 原样送入当前的模型层，会出现什么风险？

参考思路

风险在于模型层的字符级处理逻辑并未按完整单词函数名设计。也许部分情况看似能走通，但整体上会偏离当前解析器的预期输入格式，导致结果不稳定或逻辑混乱。

4.3 第二步：调度场算法如何把中缀表达式转成 RPN

在进入具体代码之前，先看整条流程图，会更容易理解每个阶段各自负责什么：

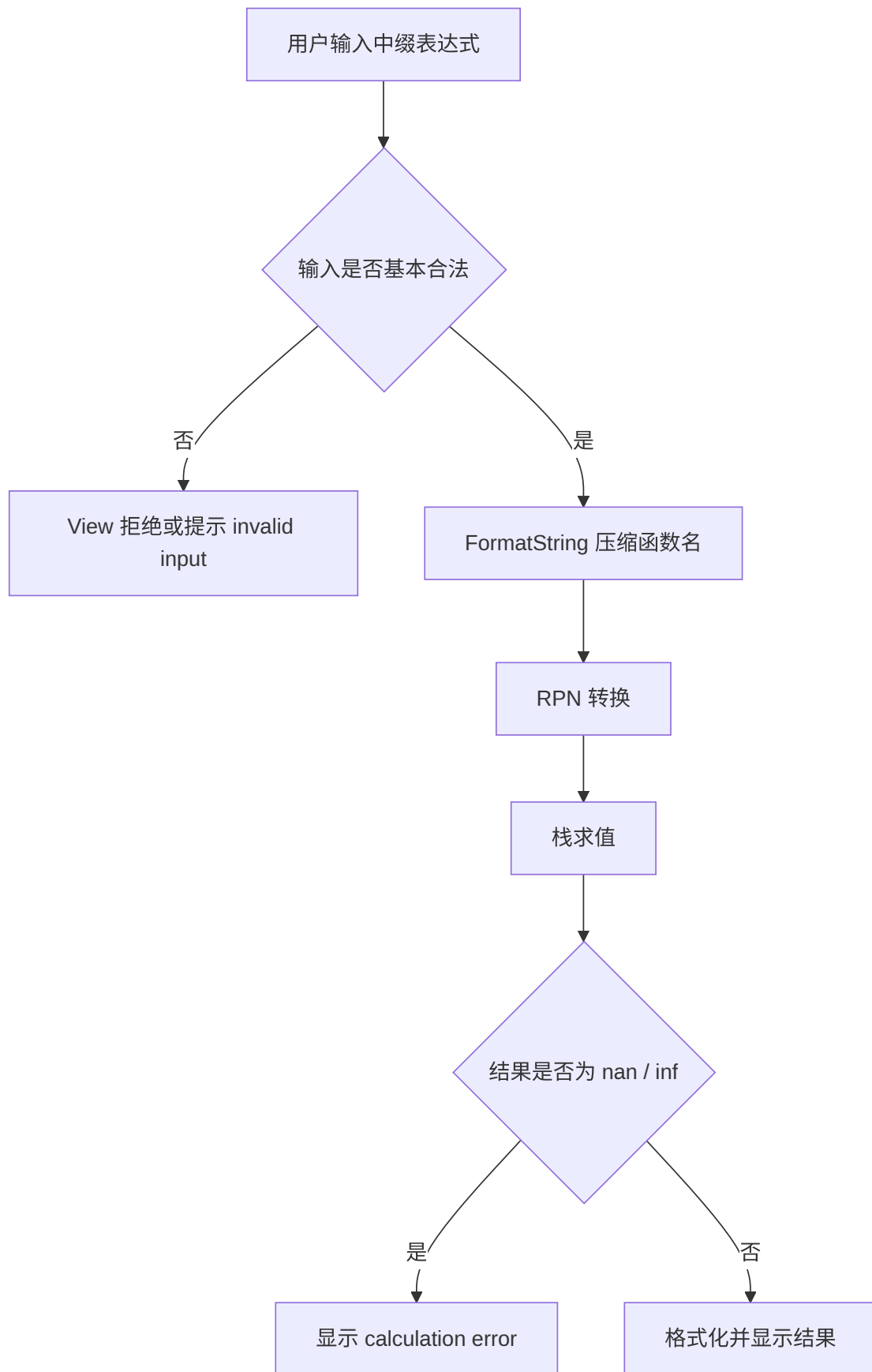


图 4.1: 表达式求值总流程

RPN 转换核心在 `src/model/reverse_polish_notation.cc`。它实现的是调度场算法 (Shunting-yard Algorithm)。先看关键代码：

```
5 void ReversePolishNotation::Convert(std::string &str) {
6     std::stack<Lexeme> operators;
7
8     size_t move_iter = 1; ///value to move the iterator
9                          ///in the string after parsing a lexeme
10
11     std::string::iterator symbol = str.begin();
12     while (symbol != str.end()) {
13         move_iter = 1;
14         if (std::isdigit(*symbol) || *symbol == 'x') {
15             move_iter = ParseNumber(symbol);
16         } else if (*symbol == '(') {
17             PushOpenParenth(operators);
18         } else if (*symbol == ')') {
19             CloseParenth(operators);
20         } else {
21             ParseOperator(operators, symbol, str);
22         }
23         symbol += move_iter;
24     }
25     while (!operators.empty()) {
26         rpn_list_.push_back(operators.top());
27         operators.pop();
28     }
29 }
30
31 size_t ReversePolishNotation::ParseNumber(std::string::iterator it) {
32     Lexeme new_lexeme;
33
34     /* check if a lexeme is a number or exponential notation or x */
35     while (std::isdigit(*it) || *it == '.' || *it == 'e' ||
36            (*it == '-' && *(it - 1) == 'e') || (*it == '+' && *(it - 1) == 'e') ||
37            *it == 'x') {
38         new_lexeme.value.push_back(*it);
39         ++it;
40     }
41     new_lexeme.priority = Priority::kPriority_0;
42     new_lexeme.type = LexemeType::kNumber;
43     rpn_list_.push_back(new_lexeme);
44     size_t move_iter = new_lexeme.value.length();
45     return (move_iter);
46 }
47
48 void ReversePolishNotation::ParseOperator(std::stack<Lexeme> &operators_stack,
49                                           std::string::iterator it,
50                                           std::string &str) {
51     Lexeme new_element(*it, GetPriority(it), LexemeType::kOperator);
52
53     /* if '+' or '-' is an unary sign, push 0 to RPN list */
```

```

54     if (IsUnary(it, str)) {
55         Lexeme add_zero('0', Priority::kPriority_0, LexemeType::kNumber);
56         rpn_list_.push_back(add_zero);
57     }
58
59     if (operators_stack.empty()) {
60         operators_stack.push(new_element);
61
62         /* if stack is not empty */
63     } else {
64         if (new_element.priority > operators_stack.top().priority) {
65             operators_stack.push(new_element);
66         } else {
67             /* if current element priority is less or equal then top element priority,
68             * pop and add top lexeme to the RPN list until '(' is met */
69             while (!operators_stack.empty() &&
70                 (new_element.priority <= operators_stack.top().priority) &&
71                 operators_stack.top().value != "(") {
72                 Lexeme top_lexeme = operators_stack.top();
73                 rpn_list_.push_back(top_lexeme);
74                 operators_stack.pop();
75             }
76             operators_stack.push(new_element);
77         }
78     }
79 }
80
81 void ReversePolishNotation::CloseParenth(std::stack<Lexeme> &operators_stack) {
82     Lexeme element;
83     while (!operators_stack.empty()) {
84         element = operators_stack.top();
85         if (element.value == "(") {
86             operators_stack.pop();
87             break;
88         }
89         rpn_list_.push_back(element);
90         operators_stack.pop();
91     }
92 }
93
94 void ReversePolishNotation::PushOpenParenth(
95     std::stack<Lexeme> &operators_stack) {
96     Lexeme parenthesis('(', Priority::kPriority_0, LexemeType::kOperator);
97     operators_stack.push(parenthesis);
98 }
99
100 Priority ReversePolishNotation::GetPriority(std::string::iterator it) {
101     Priority element_priority;
102     if (*it == '(') {
103         element_priority = Priority::kPriority_0;
104     } else if (*it == '+' || *it == '-') {
105         element_priority = Priority::kPriority_1;

```

```

106 } else if (*it == '*' || *it == '/' || *it == '%') {
107     element_priority = Priority::kPriority_2;
108 } else if (*it == '^' || *it == 'r') { /* pow and sqrt */
109     element_priority = Priority::kPriority_3;
110 } else {
111     element_priority = Priority::kPriority_4;
112 }
113 return element_priority;
114 }
115
116 bool ReversePolishNotation::IsUnary(std::string::iterator it,
117                                     std::string &str) {
118     if (*it == '+' || *it == '-') {
119         /* if the operator is first in the string */
120         if (it == str.begin()) {
121             return true;
122         }
123
124         /* if the operator follows '(' */
125         if (*(it - 1) == '(') {
126             return true;
127         }
128     }
129     return false;
130 }

```

阅读这段代码时，建议抓住四个核心动作：

1. 如果当前字符是数字或 x ，就解析成一个数值词元，直接加入输出列表；
2. 如果是左括号，就压入运算符栈；
3. 如果是右括号，就把栈中内容弹出，直到遇到左括号；
4. 如果是运算符，就根据优先级决定是直接入栈，还是先把栈顶更高或同级运算符弹出。

这里的“输出列表”就是最终的 RPN 结果，而“运算符栈”负责暂存尚未进入结果序列的运算符。

4.3.1 优先级是怎么定义的

在 `GetPriority(...)` 中，大致有以下层次：

- `kPriority_1`: $+$ 、 $-$
- `kPriority_2`: $*$ 、 $/$ 、取模运算
- `kPriority_3`: 幂运算与 r
- `kPriority_4`: 三角函数和对数函数

这意味着函数调用和一元数学函数的绑定会比普通四则运算更紧。

4.3.2 一元正负号是如何处理的

很多新手看到 `IsUnary(...)` 时会恍然大悟：原来这个项目不是发明了什么神秘的新运算符，而是用“补一个 0”的方式把一元正负号转成二元运算。

例如：

- -5 被视作 0-5
- (+2+2) 中的 +2 可以被视作 0+2

但它的判断条件也相对保守：只有在**表达式开头或左括号之后**，+/- 才会被认作一元符号。因此像 `2*-3` 这类写法不属于它最自然支持的输入风格，通常更稳妥的写法是 `2*(-3)`。

一个很值得写进脑子里的细节

当前实现对幂运算的处理更接近左结合，而不是许多数学教材里常见的右结合。换句话说，当一个表达式里连续出现多次幂运算时，系统更偏向先结合左侧，再继续向右计算。这不是“数学绝对真理”，而是这个具体实现的语义选择，因此你在读测试或扩展功能时一定要记住这一点。

节末自测 3

试着手工把表达式 `2+3*4` 转换为 RPN。你的结果应该是什么？为什么乘法会排在加法前面？

参考思路

结果应为 `2 3 4 * +`。因为扫描到 `*` 时，它的优先级高于栈中的 `+`，所以不会先把 `+` 弹出；最终输出时乘法更靠前，从而先算乘法。

4.4 第三步：栈求值怎样得到最终结果

当表达式已经变成 RPN 后，真正的数值计算就很直接了。请看 `src/model/calculation.cc`：

```
5 void Calculation::Parse(std::string &expression, long double x_value) {
6     rpn_.Convert(expression);
7
8     expression_ = rpn_.GetRpnList();
9     long double operation_result = 0;
10    std::stack<long double> numbers;
11    Lexeme lexeme;
12    long double number = 0;
13    while (!expression_.empty()) {
14        lexeme = expression_.front();
15        expression_.pop_front();
16        if (lexeme.type == LexemeType::kNumber) {
17            if (lexeme.value == "x") {
18                numbers.push(x_value);
19            } else {
20                try {
21                    number = std::stold(lexeme.value);
```

```

22         } catch (const std::out_of_range &e) {
23             calculation_result_ = NAN;
24             return;
25         }
26         numbers.push(number);
27     }
28 }
29 if (lexeme.type == LexemeType::kOperator) {
30     operation_result = Calculate(lexeme, numbers);
31     numbers.push(operation_result);
32 }
33 }
34 calculation_result_ = numbers.top();
35 numbers.pop();
36 }
37
38 long double Calculation::Calculate(s21::Lexeme &current_operator,
39                                   std::stack<long double> &numbers) {
40     long double result = 0;
41     long double a = numbers.top();
42     long double b = 0;
43     numbers.pop();
44
45     /* operations that require two operands */
46     if (current_operator.priority == Priority::kPriority_1 ||
47         current_operator.priority == Priority::kPriority_2) {
48         b = numbers.top();
49         numbers.pop();
50         if (current_operator.value == "+") {
51             result = b + a;
52         } else if (current_operator.value == "-") {
53             result = b - a;
54         } else if (current_operator.value == "*") {
55             result = b * a;
56         } else if (current_operator.value == "/") {
57             result = b / a;
58         } else if (current_operator.value == "%") { /* mod */
59             if (a == 0.0) {
60                 result = NAN;
61             } else {
62                 result = static_cast<int>(b) % static_cast<int>(a);
63             }
64         }
65
66         /* operations that require one operand (except pow) */
67     } else {
68         if (current_operator.value == "r") { /* sqrt */
69             result = sqrt(a);
70         } else if (current_operator.value == "^") { /* pow */
71             b = numbers.top();
72             numbers.pop();
73             result = pow(b, a);

```

```

74     } else if (current_operator.value == "s") { /* sin */
75         result = sinl(a);
76     } else if (current_operator.value == "c") { /* cos */
77         result = cosl(a);
78     } else if (current_operator.value == "t") { /* tan */
79         result = tanl(a);
80     } else if (current_operator.value == "S") { /* asin */
81         result = asinl(a);
82     } else if (current_operator.value == "C") { /* acos */
83         result = acosl(a);
84     } else if (current_operator.value == "T") { /* atan */
85         result = atanl(a);
86     } else if (current_operator.value == "l") { /* ln */
87         result = logl(a);
88     } else if (current_operator.value == "L") { /* log */
89         result = log10l(a);
90     }
91 }
92 return result;
93 }
94
95 long double Calculation::GetResult() { return calculation_result_; }

```

这个阶段的逻辑可以概括为：

1. 顺序读取 RPN 词元；
2. 如果是数字，就压入数字栈；
3. 如果是变量 `x`，就把当前 `x_value` 压栈；
4. 如果是运算符，就根据是一元还是二元操作，从栈里弹出 1 个或 2 个数，计算后再压回去；
5. 最终栈顶就是整个表达式的结果。

这就是为什么 RPN 特别适合计算机：它天然消除了括号和优先级的即时判断压力，剩下的只是一个线性扫描和一个栈。

4.4.1 从测试中理解实现细节

项目的单元测试写在 `src/model/tests/calculation_test.cc`，其中有几组非常值得你主动观察：

```

63 TEST(calculate, simple_7) {
64     s2l::Calculation calc;
65     std::string str = "2+(-5+1)";
66     long double x = 0;
67     calc.Parse(str, x);
68     long double result = calc.GetResult();
69     EXPECT_EQ(result, -2);
70 }
71

```

```

72 TEST(calculate, simple_8) {
73     s21::Calculation calc;
74     std::string str = "-(-5+1)";
75     long double x = 0;
76     calc.Parse(str, x);
77     long double result = calc.GetResult();
78     EXPECT_EQ(result, 4);
79 }
80
81 TEST(calculate, simple_9) {
82     s21::Calculation calc;
83     std::string str = "(-(-5+1)*3)";
84     long double x = 0;
85     calc.Parse(str, x);
86     long double result = calc.GetResult();
87     EXPECT_EQ(result, 12);
88 }
89
90 TEST(calculate, simple_10) {
91     s21::Calculation calc;
92     std::string str = "6/(7-7)";
93     long double x = 0;
94     calc.Parse(str, x);
95     long double result = calc.GetResult();
96     EXPECT_TRUE(std::isinf(result));
97 }
98
99 TEST(calculate, simple_11) {
100     s21::Calculation calc;
101     std::string str = "(((+2+2)*2))";
102     long double x = 0;
103     calc.Parse(str, x);
104     long double result = calc.GetResult();
105     EXPECT_EQ(result, 8);
106 }
107
108 TEST(calculate, simple_12) {
109     s21::Calculation calc;
110     std::string str = "(2+3/4)";
111     long double x = 0;
112     calc.Parse(str, x);
113     long double result = calc.GetResult();
114     EXPECT_NEAR(result, 2.75, LIMIT);
115 }
116
117 TEST(calculate, simple_13) {

```

这些测试至少告诉我们：

- 项目支持括号嵌套；
- 支持一元正负号；

- 除零会得到 `inf`，某些非法数值例如负数开方会得到 `nan`；
- 结果层会在界面中把 `nan/inf` 显示成 `calculation error`。

还有一个非常有教学意义的小坑：`mod` 在这里是通过把操作数先转成整数后再取模实现的，因此它更接近**整数取模**，而不是浮点数上的 `fmod` 行为。也就是说，小数参与取模时会先被截断。

理解“实现语义”很重要

学项目时不要只问“支持不支持 `mod`”，还要问“这个 `mod` 到底按什么语义实现”。同一个功能名，在不同项目里可能代表不同的精确定义。

本章小结

这一章的知识链条一定要串起来：

1. `FormatString` 负责把用户友好的函数名转换成内部友好的单字符函数码；
2. `ReversePolishNotation` 负责通过调度场算法处理优先级和括号；
3. `Calculation` 负责在 RPN 上做栈求值并得到最终结果。

本章练习

1. 复述题：请解释为什么 `FormatString` 和 RPN 转换不是同一件事。
2. 手工题：请自己构造一个“开方结果再参与一次二元运算”的例子，转成大致 RPN，并说明先算哪一步。
3. 观察题：阅读测试文件中关于幂运算的测试，说明当前实现的幂运算结合性是什么。
4. 开放题：如果你想支持绝对值函数，你觉得至少需要修改哪几个位置？

本章练习参考答案

第 1 题：前者处理输入表示，后者处理运算结构。第 2 题的关键是先把操作数和运算符整理成后缀顺序，因此会先执行开方和幂，再做加法。第 3 题：当前实现更接近左结合。第 4 题通常至少要改界面输入、`FormatString`、RPN 优先级识别以及求值分支。

Chapter 5

函数绘图：同一套计算内核如何复用到图像显示

当一个项目已经能计算 $f(x)$ 的数值结果时，绘图其实就不再神秘了。你只需要回答一个问题：如果我在很多个 x 点上重复求值，能不能把这些点连成图？SmartCalc 的绘图功能就是这样构建出来的。

5.1 图窗打开时到底发生了什么

图像绘制的入口在 `View::OpenGraphWindow()` 中。下面截取其中最关键的一段：

```
562 void View::OpenGraphWindow() {
563     /* if the window isn't open */
564     if (graph_ == nullptr) {
565         graph_ = new Graph(this);
566
567         int main_window_x = this->x();
568         int main_window_y = this->y();
569
570         connect(graph_, &Graph::finished, this, &View::GraphWindowClosed);
571
572         graph_->move(main_window_x + this->width() + 20, main_window_y);
573         graph_->show();
574
575         /* if the window is already open */
576     } else {
577         graph_->raise();
578         graph_->activateWindow();
579     }
580
581     /* if expression is valid */
582     if (open_parenthesis_clicked_ == 0 && string_to_calculate_.length() != 0 &&
583         operator_clicked_ == false) {
584         graph_->SetExpression(string_to_show_);
585
586         s21::FormatString formatted_str(string_to_calculate_);
```

```

587     std::string str_to_plot = formatted_str.GetString();
588     std::pair<std::vector<double>, std::vector<double>> coordinates =
589         controller_>CalculateGraphCoordinates(str_to_plot,
590             graph_>GetXRange());
591
592     graph_>BuildPlot(coordinates);
593
594 } else if (string_to_calculate_.length() != 0) {
595     graph_>Clear();
596     graph_>SetExpression("invalid input");
597
598     /* input string is empty */
599 } else {
600     graph_>Clear();
601 }
602 }
603
604 void View::GraphWindowClosed() { graph_ = nullptr; }

```

这段逻辑可以拆成两个阶段看：

1. **窗口管理阶段：**如果图窗还没打开，就创建 Graph 对象、设置位置并显示；如果已经打开，就把它提到前台。
2. **数据准备阶段：**如果当前表达式满足基本合法性，就把表达式交给 FormatString 和控制器，生成坐标，再交给图窗绘制。

这里再次体现出良好的职责分工：

- View 决定何时显示图窗、何时提示 invalid input；
- Controller 和 PlotGraph 决定怎样把表达式变成点集；
- Graph 决定怎样把点集画出来。

节末自测 1

为什么 OpenGraphWindow() 在真正调用绘图前，还要先检查括号计数和运算符状态？

参考思路

因为图窗绘制依赖当前表达式能被成功求值。如果表达式在界面层已经明显不完整，例如括号未闭合或末尾停在运算符上，那么继续往模型层传只会制造更混乱的错误。

5.2 绘图本质上是“批量代值”

坐标生成在 src/model/plot_graph.h 中：

```

30  /**
31  * @brief Calculates the x and y coordinates for the given expression over the

```

```

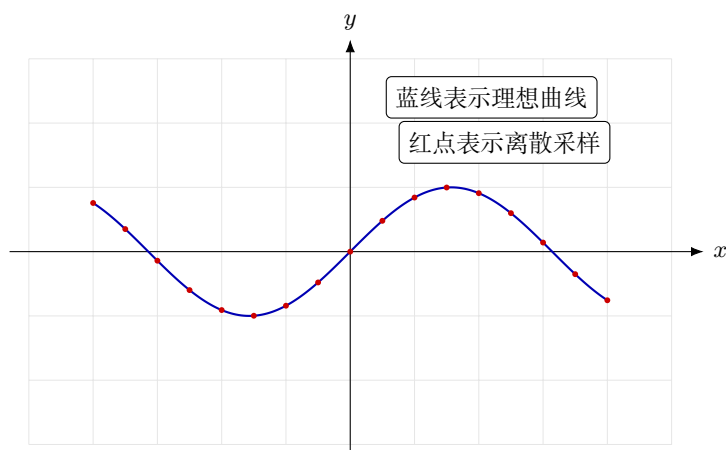
32     * specified range.
33     * @param expression string representing mathematical expression.
34     * @param x_range pair of doubles representing the range of x-values.
35     * @return pair of vectors containing the x-coordinates and corresponding
36     * y-coordinates.
37     */
38     std::pair<std::vector<double>, std::vector<double>> Calculate(
39         std::string &expression, std::pair<double, double> x_range) {
40         std::vector<double> x_vector;
41         std::vector<double> y_vector;
42
43         x_min_ = x_range.first;
44         x_max_ = x_range.second;
45
46         double y_coordinate = 0;
47         double x_coordinate = 0;
48         double step = 0.1;
49
50         for (double i = x_min_; i < x_max_; i += step) {
51             calc_expression_.Parse(expression, i);
52             y_coordinate = static_cast<double>(calc_expression_.GetResult());
53             x_coordinate = i;
54             x_vector.push_back(x_coordinate);
55             y_vector.push_back(y_coordinate);
56         }
57
58         return std::pair<std::vector<double>, std::vector<double>>{x_vector,
59                                                                 y_vector};
60     }
61
62 private:
63     long double x_min_; ///< min x-value
64     long double x_max_; ///< max x-value
65     std::pair<std::vector<double>, std::vector<double>>
66         coordinates_; ///< coordinates for plotting
67     Calculation
68         calc_expression_; ///< instance of Calculation to evaluate expressions
69

```

请一定把这段代码读懂，因为它几乎就是整个绘图功能的数学本质：

1. 读取用户给定的 x 范围；
2. 取固定步长 $step = 0.1$ ；
3. 从 x_{min} 走到 x_{max} ；
4. 每走一步，就调用同一个 `Calculation::Parse(expression, i)`；
5. 把每次得到的 (x, y) 存入两个向量里；
6. 最终把所有点交给图窗。

这意味着 SmartCalc 的绘图是**离散采样绘图**，而不是符号推导式绘图。它并不会“理解函数长什么样”，它只是“在很多点上重复算值，再把这些点显示出来”。



从这张示意图里你应该读出一个重要结论：**采样点越稀，曲线越可能显得粗糙；采样点越密，图像越平滑，但计算量也会上升。**

当前实现的现实边界

由于步长固定为 0.1，所以高频振荡函数、陡峭变化函数或者带间断的函数，图像可能不够平滑，甚至会出现“看起来不太对”的情况。这不一定是算法错了，也可能只是采样密度不够。

节末自测 2

如果把步长从 0.1 改成 0.01，最可能同时发生的两个变化是什么？

参考思路

一方面图会更平滑，另一方面计算次数会显著增多，绘图速度和资源开销也会提高。

5.3 Graph 类怎样把点真正画出来

真正把坐标送进图形控件的是 `src/view/graph.cc`：

```
15 void Graph::BuildPlot(  
16     std::pair<std::vector<double>, std::vector<double>> &coordinates) {  
17     ui_->plot->clearGraphs();  
18  
19     double x_begin = ui_->spin_box_min_x->value();  
20     double x_end = ui_->spin_box_max_x->value();  
21     double y_begin = ui_->spin_box_min_y->value();  
22     double y_end = ui_->spin_box_max_y->value();  
23  
24     QVector<double> x_vector(coordinates.first.begin(), coordinates.first.end());  
25     QVector<double> y_vector(coordinates.second.begin(),  
26                               coordinates.second.end());  
27
```

```

28     ui_>plot->xAxis->setRange(x_begin, x_end);
29     ui_>plot->yAxis->setRange(y_begin, y_end);
30
31     ui_>plot->addGraph();
32     ui_>plot->graph(0)->setData(x_vector, y_vector);
33     ui_>plot->replot();
34     ui_>plot->update();
35
36     ui_>plot->setInteraction(QCP::iRangeZoom, true);
37     ui_>plot->setInteraction(QCP::iRangeDrag, true);
38 }
39
40 void Graph::SetExpression(QString expression) {
41     ui_>expression_to_plot->setText(expression);
42 }
43
44 void Graph::Clear() {
45     ui_>expression_to_plot->clear();
46     ui_>plot->clearGraphs();
47     ui_>plot->replot();
48     ui_>plot->update();
49 }
50
51 std::pair<double, double> Graph::GetXRange() {
52     std::pair<double, double> x_range;
53     x_range.first = ui_>spin_box_min_x->value();
54     x_range.second = ui_>spin_box_max_x->value();
55     return x_range;
56 }

```

这段代码的关键工作包括：

1. 从图窗上的输入框读取显示范围 `x_begin/x_end/y_begin/y_end`;
2. 把 `std::vector<double>` 转成 `QVector<double>`;
3. 调用 `addGraph()` 和 `setData(...)` 把数据塞给 `QCustomPlot`;
4. 设置坐标轴范围并 `replot()`;
5. 打开拖拽和缩放交互。

这里你可以看到一个很清楚的分层边界：

- `PlotGraph` 只负责产出“点数据”；
- `Graph` 只负责“显示这些点”。

也正因为这样，如果未来你想把图形库从 `QCustomPlot` 换成别的库，理论上主要改的是 View 侧的显示逻辑，而不是整个表达式引擎。

5.4 为什么有时候“图画不出来”其实不是程序坏了

图像显示相关的一个常见误区是：用户以为“没看到曲线 = 绘图失败”。其实还可能几种更常见的原因：

1. 当前输入表达式不合法，所以图窗被清空并显示 `invalid input`；
2. 图像真实存在，但 `y` 轴范围太小，曲线全部跑到可视区域外；
3. 采样步长较粗，函数变化太快，看起来不平滑；
4. 表达式包含无定义区间，某些点的值为 `nan` 或 `inf`，导致显示效果不理想。

因此，绘图排错时你应该优先问：

- 表达式本身能否先通过 `=` 求值得到合理结果？
- `x` 范围和 `y` 范围是不是太极端？
- 函数是否存在明显的间断点或爆炸点？

一个很有用的实验习惯

先从容易验证的函数开始，例如 `x`、`x*x`、`sin(x)`、`log(x)`。如果这些都正常，再去尝试更复杂的表达式。这样你能更快区分“系统整体有问题”还是“某个特定函数/范围有问题”。

本章小结

到这里，你应该已经能把 SmartCalc 的绘图功能解释成一句非常朴素的话：

给定表达式和 `x` 范围，用同一套计算内核在很多个 `x` 点上重复求值，把得到的坐标交给图形控件显示。

本章练习

1. 复述题：请说明 `OpenGraphWindow()`、`PlotGraph::Calculate(...)` 和 `Graph::BuildPlot(...)` 三者的分工。
2. 动手题：如果你想让图更平滑，先去改哪一个文件、哪一个变量最直接？
3. 分析题：为什么“自动适配 `y` 轴范围”不是当前实现天然具备的能力？
4. 开放题：如果要支持用户自定义采样步长，你会把这个参数放在哪一层更合适？请说明原因。

本章练习参考答案

第 1 题：View 负责触发，PlotGraph 负责算点，Graph 负责显示。第 2 题：最直接的是 `src/model/plot_graph.h` 中的 `step`。第 3 题：因为当前图窗只读取显示范围，没有根据数据整体分布做额外统计和自动缩放逻辑。第 4 题通常更适合放在 View 提供输入、Controller 传递参数、Model 使用参数的组合设计中。

Chapter 6

测试、排错与继续扩展

学完一个项目，如果只能“照着原样运行”，那还不算真正掌握。真正有用的下一步是：你要知道它怎么验证正确性、哪里容易出错，以及如果你准备继续扩展，应该从哪些地方下手。

6.1 测试覆盖了哪些核心能力

项目的模型层测试集中在 `src/model/tests/calculation_test.cc`。它没有测试 GUI，这其实是一个很合理的教学切分：界面行为通常更依赖交互测试，而纯数学逻辑更适合先用单元测试 (Unit Test) 验证。

从测试内容看，它主要覆盖了以下几类能力：

1. 基础四则运算和括号嵌套；
2. 变量 `x` 参与的表达式；
3. 幂、平方根、三角函数、反三角函数、对数函数；
4. 科学计数法；
5. 非法结果或边界结果，例如 `inf` 与 `nan`。

在 Linux 上，模型测试可以直接这样执行：

```
1 cd src
2 make test
```

这个目标做的事并不依赖 Qt 界面，而是直接编译模型相关源文件和测试文件，再链接 GTest。正因为如此，它特别适合在“界面还没研究透之前”先验证核心计算逻辑。

节末自测 1

为什么一个带 GUI 的项目，仍然值得先把数学逻辑单独抽出来做单元测试？

参考思路

因为 GUI 测试成本更高、依赖更复杂，而数学逻辑通常可重复、可自动化、可精确比较预期结果。先把最核心的业务逻辑用单元测试稳住，后续调 GUI 才不至于两边问题混在一起。

6.2 常见坑：不仅要知道是什么，还要知道怎么定位

下面列出几个学习这个项目时最容易遇到、也最值得主动记住的问题。

6.2.1 构建层的坑

- **现象：** CMake 提示找不到 Qt。
- **可能原因：** CMAKE_PREFIX_PATH 没传对，或者传到了错误层级。
- **定位方法：** 先检查 `$HOME/Qt/6.7.0/gcc_64/lib/cmake/Qt6/Qt6Config.cmake` 是否存在。
- **解决办法：** 把 CMake 前缀路径指向 `gcc_64` 目录。
- **现象：** 执行 Makefile 某些目标时报 `open` 相关错误。
- **可能原因：** 当前目标带有 macOS 风格命令。
- **定位方法：** 直接读 `src/Makefile`，确认是命令本身不适配，而不是源码编译失败。
- **解决办法：** Linux 下把 CMake 作为主构建方式；若一定要用 Makefile，可手工改成 `xdg-open` 或直接省略打开动作。

6.2.2 表达式层的坑

- **现象：** 某些你觉得“数学上合理”的表达式在界面里输不进去，或者结果不符合你的结合顺序直觉。
- **可能原因：** 当前实现对一元正负号和幂运算结合性有自己的语义约束。
- **定位方法：** 阅读 `IsUnary(...)` 和对应测试用例。
- **解决办法：** 尽量按当前系统偏好的输入形式书写，例如用 `2*(-3)` 替代 `2*-3`。
- **现象：** `mod` 遇到小数时结果怪异。
- **可能原因：** 当前实现先把参与者转成 `int` 再取模。
- **定位方法：** 读 `calculation.cc` 中 `%` 分支。
- **解决办法：** 明确告诉自己当前语义是整数取模，而不是浮点 `fmod`。

6.2.3 绘图层的坑

- **现象：**图窗里看不到曲线。
- **可能原因：**表达式无效、坐标范围太小、函数值超出可视区域、采样过粗或函数存在间断。
- **定位方法：**先用 `=` 验证表达式，再用简单函数测试图窗，再调整 `x/y` 范围。
- **解决办法：**从最简单可验证案例逐步增加复杂度。

排错时最重要的习惯

不要同时改很多地方。每次只改一个变量，例如只换一个构建参数、只改一条表达式、只调一个坐标范围。这样你才知道究竟是哪一个因素导致结果变化。

节末自测 2

假设 `sin(x)` 可以正常绘图，但 `log(x)` 看起来像“没画出来”。你会优先检查哪些方面？

参考思路

先看 `x` 范围是否包含大量非正数，因为 `log(x)` 在这些区域无定义；再看 `y` 轴范围是否过窄；最后再考虑采样与显示问题。

6.3 如果要继续扩展，这个项目适合怎么改

一个项目是否适合作为学习材料，很大程度上取决于它是否“可扩展”。SmartCalc 在这方面其实很友好，因为模块边界已经比较清晰。

下面给出几条很自然的扩展路线：

1. **增加新函数：**例如 `abs`、`exp`。通常要同步修改界面按钮、`FormatString`、RPN 优先级判断和求值分支。
2. **改进绘图体验：**例如允许用户设置采样步长、自动计算更合适的 `y` 范围、对间断点做分段绘制。
3. **增强输入系统：**例如支持键盘直接输入、支持空格清理、支持更强健的模型层输入校验。
4. **增加历史记录：**保存最近若干次表达式和结果。
5. **改造架构：**例如把当前较重的 View 状态管理进一步拆分，使界面代码更短更清晰。

从教学角度看，最推荐的练手顺序通常是：

1. 先加一个新函数；
2. 再做一个小的绘图增强；
3. 最后再尝试界面级功能，例如历史记录或键盘输入。

原因很简单：先从“局部、确定、可测试”的修改开始，更容易获得正反馈。

6.4 全文复习与综合练习

到这里，你应该已经可以把整个项目讲成一个完整故事：

这是一个基于 Qt 的科学计算器项目。它通过 MVC 把界面与数学逻辑分离；用户输入的表达式会先经过格式压缩，再用调度场算法转成逆波兰序列，然后通过栈求值得到结果；绘图功能复用了同一套计算核心，对多个 x 点重复代值，再把坐标交给 QCustomPlot 显示。

下面给出一组综合练习，帮助你把整条链路串起来。

综合练习

- 1. 总复述题：**不看源码，完整说明从点击 = 按钮到结果显示的处理流程，至少提到 View、FormatString、Controller、RPN、Calculation 五个角色。
- 2. 动手题：**在本地运行项目，分别输入 $x*x$ 、 $\sin(x)$ 、 $\log(x)$ 三个表达式观察绘图结果，并记录每个函数在什么 x/y 范围下更容易看清特征。
- 3. 代码分析题：**阅读 view.cc 中与 BackspaceClicked() 相关的逻辑，解释为什么回退功能看似简单，实际上需要理解多种输入状态。
- 4. 扩展设计题：**设计“支持绝对值函数”的实现计划。请列出你准备修改的文件，以及每个文件里要改的职责点。
- 5. 期末风格开放题：**如果你要把这个项目做成一份可展示的课程作业，你会优先补哪三项改进？请从“用户体验”“可维护性”“可验证性”三个角度分别回答。

综合练习参考解答方向

第 1 题应能把整条调用链说顺。第 2 题没有唯一数值答案，但应能观察到 $\log(x)$ 对定义域更敏感。第 3 题的关键在于：回退不仅要删字符，还要恢复多种状态标志。第 4 题通常至少涉及 View、FormatString、RPN/优先级与求值分支。第 5 题示例：用户体验可做键盘输入和历史记录，可维护性可拆分 View 状态逻辑，可验证性可补更多模型测试和边界测试。

本章小结

最后请记住，真正完成一个项目学习，不是把每一行代码都背下来，而是获得三种能力：

1. 我能独立编译、运行并复现实验；
2. 我能解释模块边界和关键数据流；
3. 我能在这个基础上做小幅但正确的扩展。

如果你已经能做到这三点，那么 SmartCalc 对你来说就不再只是一个“别人写好的仓库”，而是真正变成了你自己的学习资产。

附录 A

附录：项目中定义的类总览

这一份附录专门服务于一种很常见的学习需求：我已经大致知道项目在做什么，但一看到这么多类和头文件，还是会分不清谁是谁。因此本附录不追求“每个成员函数逐行解释”，而是重点回答四个问题：

1. 这个类属于哪一层；
2. 它的主要职责是什么；
3. 它依赖谁、被谁调用；
4. 学习或扩展时，什么时候应该优先看它。

如何使用这份附录

最推荐的用法不是从头到尾死读，而是在你阅读正文某一章时，把这里当作“类索引表”。例如你正在理解表达式引擎，就重点看 `FormatString`、`ReversePolishNotation`、`Lexeme` 和 `Calculation`；如果你正在理解界面交互，就重点看 `View` 和 `Graph`。

A.1 类之间的整体关系

先用一句话概括整张类关系图：**Qt 窗口类负责交互，控制器负责转发，模型类负责表达式和绘图数据，底层辅助类负责词法与格式转换。**

除了这张偏“模块关系”的概览图之外，附录里再给出一张更偏工程视角的 UML 类图。它的目标不是展示每一个私有函数细节，而是让你快速看清三件事：谁继承了 Qt 的基类、谁持有谁、谁依赖谁。

如果你以前几乎没读过 UML，也不用紧张。你可以先把这张图当作一种“面向对象关系地图”，它并不是要你一次看懂所有符号，而是帮助你快速建立整体印象。

这里先给出一个非常实用的读图口诀：**先看框，再看线，最后看箭头方向。**

- **框：**每个矩形框代表一个类。框里通常会分成“类名”“成员变量”“成员函数”几部分。
- **正号与负号：** + 通常表示公开成员，- 通常表示私有成员。对当前阶段来说，你只需要知道它们在表达“这个成员对外是否直接开放”。

- **空心三角箭头：**通常表示继承关系。比如 View 指向 QMainWindow，意思是 View 是从它继承来的。
- **普通箭头：**可以先把它理解成“一个类会调用、依赖或使用另一个类”。在这份图里，我们主要用它表达谁依赖谁。
- **从上到下的布局：**这次我把 UML 图改成了竖直方向，更适合按层往下看：上层通常更接近界面与 Qt 基类，往下逐步进入控制层、模型层和底层辅助结构。

你第一次读这张图时，完全不需要把每个成员函数都看一遍。更高效的办法是只回答三个问题：

1. 哪些类属于界面层，哪些类属于模型层？
2. 谁继承了 Qt 提供的窗口基类？
3. 从用户点击按钮开始，依赖关系大致会往哪个方向传下去？

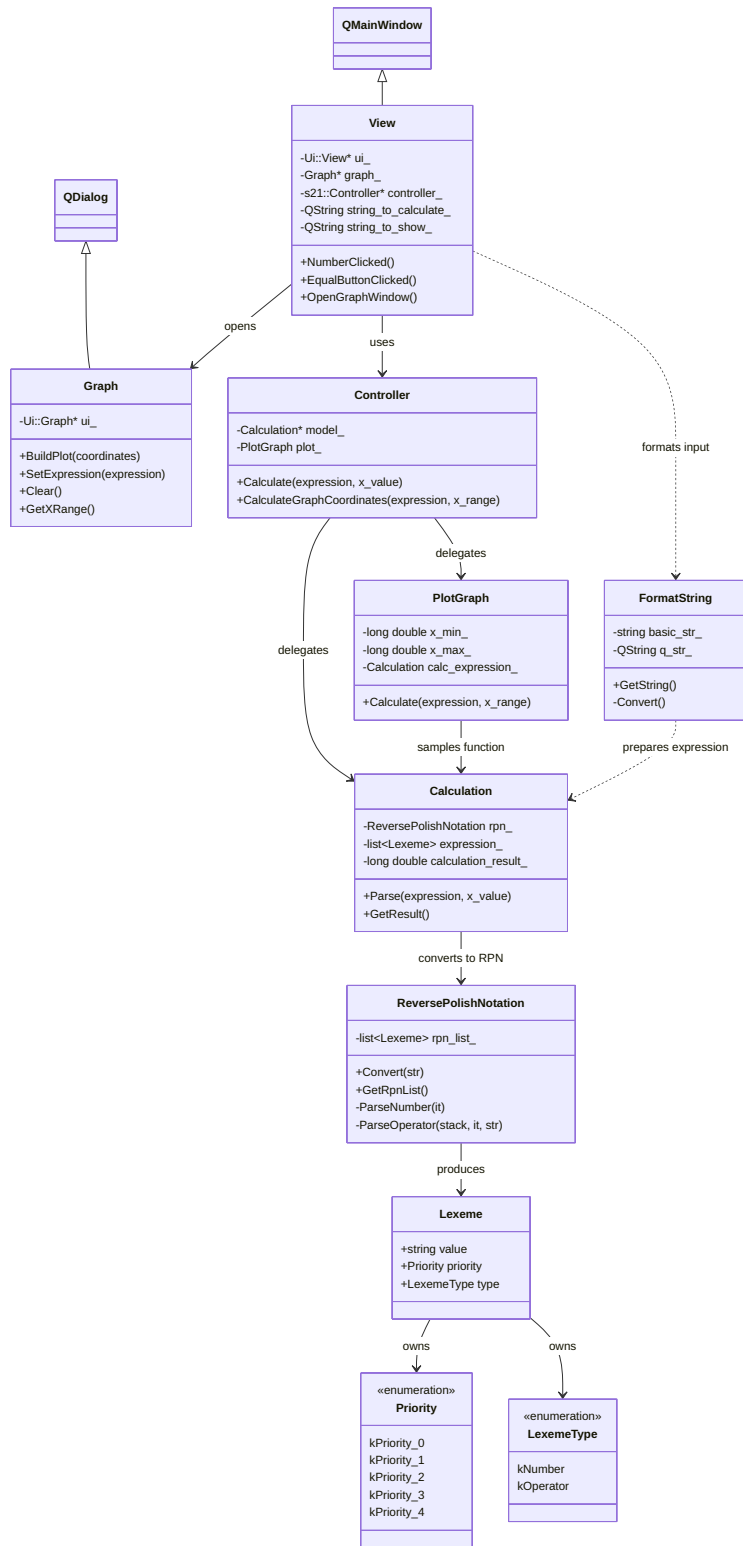


图 A.1: SmartCalc 项目 UML 类图

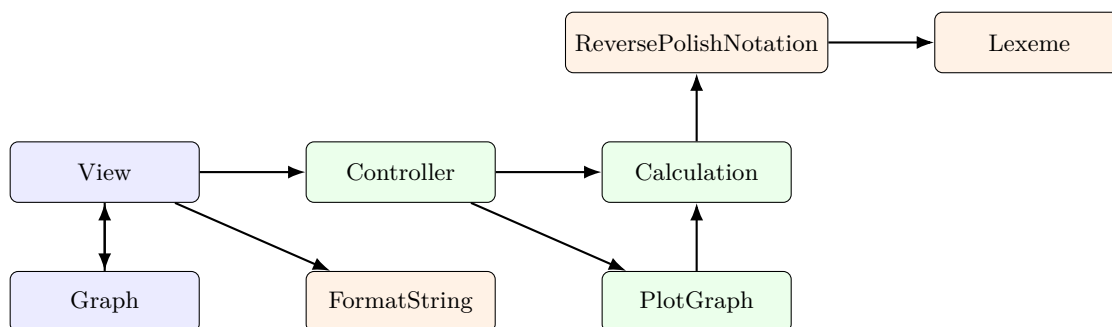
阅读这张 UML 图时，建议按下面这个顺序观察：

1. 先看最上方的 Qt 基类和界面类：View 继承自 QMainWindow，Graph 继承自 QDialog；
2. 再往下看控制层：Controller 站在界面和模型之间，负责转发请求；

3. 再看模型主干: `Calculation`、`PlotGraph`、`ReversePolishNotation`、`Lexeme` 这些类逐层向下组织表达式计算逻辑;
4. 最后看横向依赖: `View` 依赖 `Controller` 与 `FormatString`, 而绘图与表达式解析又会继续向模型核心收敛。

第一次读 UML 时的正确目标

你的第一目标不是“逐个成员背下来”，而是先回答：这个类大概属于哪一层、它和谁关系最近、如果我要改某个功能应该先去看谁。只要这三点建立起来，这张 UML 图就已经发挥作用了。



A.2 界面层类

A.2.1 View

所在文件： `src/view/view.h`、`src/view/view.cc`

它是什么： 整个计算器主窗口类，继承自 `QMainWindow`。

主要职责：

- 接收按钮点击、括号输入、小数点、变量、函数按钮等用户操作；
- 维护当前显示字符串与内部计算字符串；
- 通过一组状态标志约束输入合法性；
- 在用户点击 `=` 或打开绘图窗口时，把请求交给控制器；
- 把模型返回的结果整理成适合显示的形式。

为什么它重要：

很多初学者会低估 `View`，以为它只是“放按钮的地方”。其实在这个项目里，`View` 还承担了大量输入约束逻辑，例如防止重复小数点、控制括号平衡、限制运算符出现位置等。所以它既是界面层，也是交互规则最集中的地方。

什么时候优先看它：

当你想理解“用户点击按钮后究竟发生了什么”时，`View` 是第一入口；当你想加新按钮、新输入行为或修改显示规则时，也应该优先看它。

快速识别题：View

如果你想新增一个按钮，例如“历史记录”或“绝对值函数”，你觉得最先需要改哪个类？为什么 View 往会是第一站？

参考思路

因为按钮首先属于界面交互的一部分。即使后续还要修改模型逻辑，界面层依然通常是最先暴露新功能入口的位置。

A.2.2 Graph

所在文件： `src/view/graph.h`、`src/view/graph.cc`

它是什么： 绘图对话框类，继承自 `QDialog`。

主要职责：

- 显示表达式对应的图像；
- 从图窗控件读取当前的坐标范围；
- 把 `std::vector<double>` 形式的坐标转换给 `QCustomPlot` 使用；
- 提供清空图像、更新曲线、显示当前表达式等操作。

为什么它重要：

Graph 把“算点”和“画图”这两件事分开了。它不负责表达式求值本身，而是负责把已有点集显示出来，这种分层使得绘图逻辑更容易维护。

什么时候优先看它：

当你要调整图像显示效果、坐标轴范围、缩放拖拽、曲线显示样式时，应该优先看它；当你只是想改变采样方式，则应去看 `PlotGraph`。

A.3 控制层类

A.3.1 Controller

所在文件： `src/controller/controller.h`

它是什么： MVC 中的控制器，负责连接界面层和模型层。

主要职责：

- 接收来自 View 的“计算表达式”请求；
- 接收来自 View 的“生成绘图坐标”请求；
- 把这些请求转交给 `Calculation` 或 `PlotGraph`。

为什么它重要：

它虽然代码很薄，但承担了边界管理职责：界面层不用直接理解模型内部细节，模型层也不用感知任何 Qt 控件。对于教学来说，这种“薄但清晰”的控制器特别值得体会。

什么时候优先看它：

当你刚开始梳理调用链、想快速知道界面如何触发模型时，先看控制器最省力。

A.4 模型层类

A.4.1 Calculation

所在文件： `src/model/calculation.h`、`src/model/calculation.cc`

它是什么：项目的核心求值类。

主要职责：

- 接收待计算表达式与 x 的取值；
- 调用 `ReversePolishNotation` 把表达式转换成后缀序列；
- 使用数字栈按 RPN 规则完成求值；
- 处理四则运算、幂、平方根、三角函数、对数函数和变量替换。

为什么它重要：

如果把整个项目比作一台机器，`Calculation` 就是最核心的计算引擎。无论是点击 `=` 还是绘图采样，底层都绕不开它。

什么时候优先看它：

当你想验证表达式到底是怎么得到结果的，或者想增加新运算符、新函数时，它是必看的核心类。

A.4.2 PlotGraph

所在文件： `src/model/plot_graph.h`

它是什么：用于生成函数曲线坐标的模型类。

主要职责：

- 接收表达式和 x 范围；
- 以固定步长遍历 x ；
- 对每个采样点调用 `Calculation` 计算对应的 y ；
- 返回一组 x/y 向量供图窗显示。

为什么它重要：

它揭示了本项目绘图的本质：不是做符号推导，而是做离散采样。理解了 `PlotGraph`，你就真正理解了这个项目的绘图功能。

什么时候优先看它：

当你想调整采样步长、优化绘图精度、引入自适应采样，或者解释“为什么某些图看起来不够平滑”时，它是第一目标。

A.4.3 ReversePolishNotation

所在文件： `src/model/reverse_polish_notation.h` 与 `src/model/reverse_polish_notation.cc`

它是什么： 把中缀表达式转换成逆波兰表示法的类。

主要职责：

- 顺序扫描输入字符串；
- 识别数字、变量、括号和运算符；
- 用运算符栈处理优先级和括号；
- 生成一个按后缀顺序组织的 `Lexeme` 列表。

为什么它重要：

它把“如何理解表达式结构”这件事从真正的数值计算中拆了出来。没有这一步，`Calculation` 就不得不一边处理优先级、一边求值，逻辑会混乱得多。

什么时候优先看它：

当你想理解调度场算法在工程里是怎样落地的，或者准备新增一元/二元运算符时，它非常关键。

A.4.4 Lexeme

所在文件： `src/model/lexeme.h`

它是什么： 表达式中的“最小语义单元”类，也就是词元类。

主要职责：

- 保存一个词元的值；
- 记录其优先级；
- 区分它是数字还是运算符。

为什么它重要：

它让表达式不再只是“裸字符串”，而是变成带有语义信息的小对象序列。这样 RPN 转换和求值阶段都能写得更清楚。

什么时候优先看它：

当你发现自己总在问“程序是怎么区分数字和运算符的”，或者“优先级信息到底存在哪”，就应该回来看这个类。

A.5 辅助类

A.5.1 FormatString

所在文件： `src/view/format_string.h` 与 `src/view/format_string.cc`

它是什么： 把界面层的 `QString` 表达式转换成更适合模型解析的内部字符串的辅助类。

主要职责：

- 把 Qt 的 `QString` 转成标准库字符串；
- 把 `sin`、`cos`、`sqrt` 这类较长函数名压缩成模型层更容易处理的单字符表示；
- 让后续解析器面对更统一的输入格式。

为什么它重要：

它是一座“界面友好表示”和“内部机器友好表示”之间的桥。很多人第一次读项目时会忽略它，但实际上它是表达式引擎链路里的关键预处理步骤。

什么时候优先看它：

当你想增加新数学函数、理解为什么模型层不直接解析完整英文函数名、或者打算调整输入表达式格式时，应该优先看它。

A.6 一个很重要但不是项目自定义的类：QCustomPlot

严格说来，`QCustomPlot` 不是本项目自己定义的类，而是第三方绘图库源码被直接放进了仓库的 `src/view/qcustomplot.h` 和 `src/view/qcustomplot.cc`。之所以这里专门提一句，是因为很多初学者第一次看到这个超长头文件会误以为“这也是我必须立刻看懂的项目代码”。

其实不是。

对学习 `SmartCalc` 来说，你只需要知道：

- 它负责真正把点数据画成图；
- 项目中的 `Graph` 类只是调用它的接口；
- 在没有特别需求的情况下，你完全不必先去深入阅读它的内部实现。

阅读优先级建议

如果你现在的目标是学会这个项目，而不是研究绘图库本身，请把阅读顺序放在 `View`、`Controller`、`Calculation`、`ReversePolishNotation`、`PlotGraph` 这些项目自定义类上。`QCustomPlot` 可以先当作“已经可用的画图引擎”。

附录小结

这份附录最想帮你建立的，不是“背出每个类名”，而是形成一种阅读源码的定位能力：

1. 遇到按钮点击、显示和输入限制问题，看 `View`；
2. 遇到界面和模型之间的调用关系，看 `Controller`；
3. 遇到表达式求值问题，看 `Calculation` 与 `ReversePolishNotation`；
4. 遇到绘图采样问题，看 `PlotGraph`；
5. 遇到输入字符串如何变成内部格式的问题，看 `FormatString`；
6. 遇到“这个最小词元对象到底存了什么”的问题，看 `Lexeme`。

如果你能做到按问题定位到相应类，那么这份项目代码对你来说就已经不再“杂乱”，而是开始变成一张能被你主动导航的地图。